

Assignment I: Real-Time Data Engineering and Workflow Orchestration

DA5402W: MLOps Lab

Total Marks: 60

Submission Deadline: [29th June, 2026]

The assignment consists of three independent parts. Each part should be implemented and submitted separately.

Part A: Kafka-Based Data Ingestion (20 Marks)

A Kafka producer script is provided to run to get the data stream.

Tasks

1. Create a Kafka topic named:

`sensor_<rollno>`

2. Configure the topic with:

- 3 partitions
- Replication factor = 1

3. Execute the provided producer and verify that records are being published successfully.

4. Consume the records from the topic and verify correct delivery.

5. Collect and report the following metrics:

- Total number of records produced
- Total number of records consumed
- Number of records received from each partition
- Producer throughput (records/sec)
- Consumer throughput (records/sec)

6. Demonstrate the effect of varying the number of consumers:

- One consumer
- Two consumers
- Four consumers

7. Create two different consumer groups and demonstrate that each group consumes the stream independently.

Discussion

Briefly discuss:

1. The role of partitions in Kafka.
2. The relationship between partitions and consumer parallelism.
3. The effect of increasing the number of consumers beyond the number of partitions.

Submission

Submit:

- `kafka.py`
- A brief report containing:
 - Configuration details
 - Results
 - Discussion

Part B: Stream Processing using Spark Structured Streaming (20 Marks)

Using Spark Structured Streaming, implement a real-time preprocessing and analytics pipeline for the sensor stream obtained from Kafka.

Data Preprocessing

1. Define an appropriate schema for the incoming records.
2. Print the schema .
3. Handle missing values:
 - Replace missing temperature values using the average temperature of the same sensor over the previous 5-minute window, OR report and drop records if insufficient history exists.
4. Remove duplicate records using:

`sensor_id + timestamp`

5. Identify and remove invalid records:
 - Temperature < -20
 - Temperature > 100
 - Invalid timestamps
6. Convert timestamp strings into Spark Timestamp format.
7. Create the following features:
 - Hour of day
 - Day of week
 - Weekend indicator

Streaming Analytics

8. Compute the average temperature per sensor.
9. Compute the maximum temperature per sensor.
10. Compute the number of active sensors as sensors that have transmitted at least one record within the last 5 minutes.
11. Compute the distribution of sensor status values.

Event-Time Processing

12. Implement a 5-minute tumbling window and compute the average temperature for each window.
13. Apply watermarking with a delay threshold of 5 minutes.
14. Inject late events and demonstrate: (i) accepted records (ii) discarded records

Reporting

Generate a summary table containing:

Metric	Value
Missing Values Corrected	
Duplicate Records Removed	
Invalid Records Removed	
Late Records Accepted	
Records Discarded by Watermarking	

Performance Analysis

Collect and report:

- Input rate (rows/sec)
- Processing rate (rows/sec)
- Batch duration
- State store size (if applicable)

Submission

Submit:

- `spark_streaming.py`
- A brief report containing:
 - Summary Table
 - Analytics results
 - Event-time processing results (Plots Preferred)
 - Performance metrics

Part C: Workflow Orchestration using Apache Airflow (20 Marks)

Design and implement an Apache Airflow DAG to automate a data processing workflow.

Workflow Tasks

The DAG must contain the following tasks:

1. DataGeneration
2. DataValidation
3. DataPreprocessing
4. FeatureEngineering
5. Analytics
6. ReportGeneration

Requirements

1. Design an Airflow DAG using the tasks listed above.
2. Determine appropriate dependencies between tasks.
3. Identify tasks that can execute in parallel and implement parallel execution wherever appropriate.
4. Organize related tasks using at least one `TaskGroup`.
5. Configure retry policies:
 - Retries = 3
 - Retry delay = 1 minute
6. Use a `BranchPythonOperator` to implement conditional execution.
7. Assign suitable priority weights to selected tasks.
8. Schedule the DAG to execute every 5 minutes.
9. Monitor DAG execution using the Airflow UI.

Airflow Report Requirements

Include:

1. DAG graph screenshot.
2. Description of task dependencies.
3. Explanation of parallel execution opportunities.
4. Branching behavior screenshot.
5. Retry execution screenshot.
6. Task duration summary.

Discussion

Briefly discuss:

1. The purpose of workflow orchestration.
2. The benefits of retries in workflow execution.
3. The advantages of parallel task execution.
4. The role of branching in workflow design.

Submission

Submit:

- `airflow_dag.py`
- A brief report containing:
 - DAG design
 - Dependency analysis
 - Screenshots
 - Discussion

Final Submission

Submit a single ZIP archive containing:

- `consumer.py`
- `spark_streaming.py`
- `airflow_dag.py`
- Kafka report
- Spark Structured Streaming report
- Airflow report